

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

كورس ال Data Structure

اعداد : معتز عامر عبد الفتاح

MOATAZ AL ZOZ

الدرس الاول :- pointer

```
#include <iostream>
using namespace std;

int main()
{
    int x = 20;
    // لطباعة قيمة (اكس)
    cout << x << endl; // 20
    // لطباعة عنوان المكان المتخزن فيه المتغير (اكس)
    cout << &x << endl; // 0x61ff0c
    // لوضع عنوان المكان المتخزن فيه المتغير (اكس) في متغير
    int *p = &x;
    // لطباعة قيمة (اكس) من خلال العنوان المتخزنة فيه
    cout << *p << endl; // x = 20
    // لتعديل قيمة (اكس) من خلال العنوان
    *p = 50;
    cout << *p << endl; // x = 50
    // متغير ليس له اسم مقدس استدعية إلا بعنوان المكان المتخزن فيه
    int *b = new int(20);
    cout << *b << endl;
    return 0;
}
```

الدرس الثاني :- structure

```
#include <iostream>
using namespace std;
// structure
struct Book
{
    string name;
    string author;
    int pag;
    int price;
};
int main()
{
    Book op1;
    op1.name = "book1";
```

```

op1.author = "author1";
op1.pag = 300;
op1.price = 50;
cout << op1.name << endl;
cout << op1.author << endl;
cout << op1.pag << endl;
cout << op1.price << endl;
Book op2 {"book2" , "author2" , 305 , 55};
return 0;
}

```

الدرس الثالث :- union

```

#include <iostream>
using namespace std;
// اليونيون ييخزن في جميع المتغيرات اخر قيمة دخلت
union Boox
{
    double weight;
    double height;
};
int main()
{
    Boox op1;
    op1.weight = 20; // 10
    op1.height = 10; // 10
    cout << op1.weight << endl;
    cout << op1.height << endl;
    return 0;
}

```

الدرس الرابع :- Linkd List

```

#include <iostream>
using namespace std;
// دة الداتا تايب الكبير اللي هيحجز المكانين
// في الميموري مكان للداتا ومكان هيشاور على النود الثانية
struct node
{
    int data; // دة المكان الخاص بالداتا
    node * next; // ودة المكان اللي هيشاور على النود الثانية
};
// هنا انا هعمل البوينتر اللي هيشاور على اول عنصر في اللينكد ليست

```

```

node * head = NULL;
// الدالة دي هتضيف نود جديدة للينكد ليست والتود دي هيكون
// ليها قيمة هتتخزن في الداتا ف الدالة دي هتأخذ قيمة من نفس نوع
// البيانات بتاعة الداتاعلشان تخزنها في النود وتشاور عليها
// وهتعمل الربط بين النود واللينكد ليست
void insert_Node (int value);

int main()
{
    // اللينكد ليست
    insert_Node(5);
    insert_Node(10);
    insert_Node(15);
    insert_Node(7);
    return 0;
}

void insert_Node (int value)
{
    // البوينتر ده هيشاور على الداتا تايب اللي اسمه نود
    node * new_node , * last;
    // لعمل نود جديد بيجز قيمتين
    // node n1 = القيمة;
    new_node = new node ;
    // هنا انا عطيت قيمة لجزء الداتا اللي في النود
    new_node->data = value;
    /*الربط بين النود واللينكد ليست */
    // هنا خلّيت الهيد بتشاور على النود إذا كانت أول نود في اللينكد ليست
    if (head == NULL)
    {
        head = new_node;
        // خلّيت الجزء الحاص بالنيكست مش بيساوي حاجة
        new_node->next = NULL;
    }
    else
    {
        last = head;
        while (last->next != NULL)
        {
            // هتخلية يشاور على النود اللي بعده
            last = last->next;
        }
        // خلي آخر عنصر في النود العنصر اللي قبلية بيشاور عليه
        last->next = new_node;
        // خلي الجزء لخاص بالنيكست مش بيشاور على آخر نود
        new_node->next = NULL;}}

```

الدرس الخامس :- متابعة دروس

linked list الجزء الخاصة بعمل

دالة لطباعة النودز في linked

list

```
#include <iostream>
using namespace std;
// دة الداتا تايب الكبير اللي هيحجز المكانين
// في الميموري مكان للداتا ومكان هيشاور على النود الثانية
struct node
{
    int data; // دة المكان الخاص بالداتا
    node * next; // ودة المكان اللي هيشاور على النود الثانية
};
// هنا انا هعمل البوينتر اللي هيشاور على اول عنصر في اللينكد ليست
node * head = NULL;
// الدالة دي هتضيف نود جديدة للينكد ليست والنود دي هيكون
// ليها قيمة هتخزن في الداتا ف الدالة دي هتاخذ قيمة من نفس نوع
// البيئات بتاعة الداتا علشان تخزنها في النود وتشاور عليها
// و هتعمل الربط بين النود واللينكد ليست
void insert_Node (int value);
void display ();

int main()
{
    // اللينكد ليست
    insert_Node(5);
    insert_Node(10);
    insert_Node(15);
    insert_Node(7);
    display();
    return 0;
}

void insert_Node (int value)
{
    // البوينتر دة هيشاور على الداتا تايب اللي اسمه نود
    node * new_node , * last;
```

```

// لعمل نود جديد بيجز قيمتين
// node n1 = القيمة;
new_node = new node ;
// هنا انا عطيت قيمة لجزء الداتا اللي في النود
new_node->data = value;
/* الربط بين النود واللينكد ليست */
// هنا خليت الهيد بتساور على النود إذا كانت اول نود في اللينكد ليست
if (head == NULL)
{
    head = new_node;
    // خليت الجزء الحاص بالنيكست مش بيساوي حاجة
    new_node->next = NULL;
}
else
{
    last = head;
    while (last->next != NULL)
    {
        // هتخلية بتساور على النود اللي بعده
        last = last->next;
    }
    // خلي اخر عنصر في النود العنصر اللي قبلية بتساور عليه
    last->next = new_node;
    // خلي الجزء لخاص بالنيكست مش بيساور على حاجة في اخر نود
    new_node->next = NULL;
}
}
//*****//
void display()
{
    node * current_node;
    if (head == NULL)
    {
        cout << "linked list is empty.\n";
    }
    else
    {
        current_node = head;
        while (current_node != NULL)
        {
            cout << current_node->data << "\t";
            current_node = current_node->next;
        }
    }
}
}

```

الدرس السادس :- متابعة دروس

linked list الجزء الخاص بحذف

نود محددة

```
#include <iostream>
using namespace std;
{
    int data;    // دة المكان الخاص بالداتا
    node * next; // ودة المكان اللي هيشاور على النود التالية
};
// هنا انا هعمل البوينتر اللي هيشاور على اول عنصر في اللينكد ليست
node * head = NULL;
void insert_Node (int value);
void display ();
void deleteNode (int delete);
int main()
{
    // اللينكد ليست
    insert_Node(5);
    insert_Node(10);
    insert_Node(15);
    insert_Node(7);
    display();
    deleteNode(15);
    display();
    return 0;
}
void insert_Node (int value)
{
    node * new_node , * last;
    new_node = new node ;
    new_node->data = value;
    if (head == NULL)
    {
        head = new_node;
        new_node->next = NULL;
    }
}
```

```

else
{
    last = head;
    while (last->next != NULL)
    { last = last->next;}
    last->next = new_node;
    new_node->next = NULL;
}
}
//*****
// دالة خاصة بالطباعة
void display()
{
    node * current_node;
    if (head == NULL)
    { cout << "linked list is empty.\n";}
    else
    {
        current_node = head;
        while (current_node != NULL)
        {
            cout << current_node->data << "\t";
            current_node = current_node->next;
        }
        cout << endl;
    }
}
// دالة خاصة بحذف نود مخصصة من اللينكد ليست
void deleteNode (int dilet)
{
    node * current , * previous;
    current = head;
    previous = head;
    // لو النود اللي هتتحذف هيا اول نود
    if (current->data == dilet)
    {
        head = current->next;
        free(current); // امر لحذف النود
    }
    // لو مش اول نود
    while (current->data != dilet)
    {
        previous = current;
        current = current->next; }
    previous->next = current->next;
    free(current);}

```


الدرس السابع :- متابعة دروس

linked list الدالة الخاصة

بإضافة عناصر في اول linked list

```
#include <iostream>
using namespace std;
struct node
{
    int data ;
    node * next;
};
node * head = NULL;
void insert_Node (int value);
void display ();
void insert_beg (int value);
int main()
{
    // linked list
    insert_Node(10);
    insert_Node(15);
    display();
    insert_beg(5);
    display();
    return 0;
}
void insert_Node (int value)
{
    node * new_node , * last;
    new_node = new node ;
    new_node->data = value;
    if (head == NULL)
    {
        head = new_node;
        new_node->next = NULL;
    }
    else
    {
        last = head;
        while (last->next != NULL)
```

```

        { last = last->next;}
        last->next = new_node;
        new_node->next = NULL;
    }
}
//*****//
// دالة خاصة بالطباعة
void display()
{
    node * current_node;
    if (head == NULL)
    {
        cout << "linked list is empty.\n";
    }
    else
    {
        current_node = head;
        while (current_node != NULL)
        {
            cout << current_node->data << "\t";
            current_node = current_node->next;
        }
    }
    cout << endl;
}
// دالة خاصة بإضافة عنصر في اول اللينكد ليست
void insert_beg (int value)
{
    node * new_node = new node;
    new_node->data = value;
    new_node->next = head;
    head = new_node;
}

```

الدرس الثامن:- متابعة دروس

linked list

اول نود في linked list

```

#include <iostream>
using namespace std;

```

```

struct node
{
    int data ;
    node * next;
};
node * head = NULL;
void insert_Node (int value);
void display ();
void delete_beg ();

int main()
{
    // اللينكد ليست
    insert_Node(5);
    insert_Node(10);
    insert_Node(15);
    display();
    delete_beg();
    display();
    return 0;
}

void insert_Node (int value)
{
    node * new_node , * last;
    new_node = new node ;
    new_node->data = value;
    if (head == NULL)
    {
        head = new_node;
        new_node->next = NULL;
    }
    else
    {
        last = head;
        while (last->next != NULL)
        { last = last->next; }
        last->next = new_node;
        new_node->next = NULL;
    }
}

//*****//
// الطباعة
void display()
{
    node * current_node;
    if (head == NULL)

```

```

{ cout << "linked list is empty.\n"; }
else
{
    current_node = head;
    while (current_node != NULL)
    {
        cout << current_node->data << "\t";
        current_node = current_node->next;
    }
    cout << endl;
}
// دالة خاصة بحذف نود من اول اللينكد ليست
void delete_beg()
{
    if (head == NULL)
    { cout << "Linked List is Empty. \n"; }
    else
    {
        node * first_node = head;
        head = first_node->next;
        free(first_node);}
}

```

الدرس التاسع:- متابعة دروس

linked list

الداالة الخاصة بحذف

اخر نود في linked list

```

#include <iostream>
using namespace std;
struct node
{
    int data ;
    node * next;
};
node * head = NULL;
void insert_Node (int value);
void display ();
void delete_end();
int main()
{
    // linked list
}

```

```

    insert_Node(5);
    insert_Node(10);
    insert_Node(15);
    display();
    delete_end();
    display();
    return 0;}

void insert_Node (int value)
{
    node * new_node , * last;
    new_node = new node ;
    new_node->data = value;
    if (head == NULL)
    {
        head = new_node;
        new_node->next = NULL;
    }
    else
    {
        last = head;
        while (last->next != NULL)
        { last = last->next;}
        last->next = new_node;
        new_node->next = NULL;}}

//*****//
// دالة خاصة بالطباعة
void display()
{
    node * current_node;
    if (head == NULL)
    { cout << "linked list is empty.\n"; }
    else
    {
        current_node = head;
        while (current_node != NULL)
        {
            cout << current_node->data << "\t";
            current_node = current_node->next;}}
    cout << endl;}

// دالة خاصة بحذف اخر عنصر في اللينكد ليست
void delete_end()
{
    // في حالة مفيش اي عناصر
    if (head == NULL)
    { cout << "linked list is empty \n"; }

```

```
// في حالة فيه عنصر واحد
else if (head->next == NULL)
{
    delete(head);
    head = NULL;
}
// في حالة فيه اكثر من عنصر
else
{
    node * ptr = head;
    while (ptr->next->next != NULL)
    { ptr = ptr->next;}
    delete (ptr->next);
    ptr->next = NULL; }}
```

الدرس العاشر خاص ب ثاني نوع وهوا

STAKE

```
#include <iostream>
using namespace std;
// تطبيق عملي على الاستاك
#define size 5//حجم سابت
int stake [size];//
int top = -1;

void push (int value);// اضافة عناصر
int pop ();// حذف اخر عنصر
int peek ();// عرض اخر عنصر
void display();// عرض كل العناصر
int main ()
{
    // stake
    push (5);
    push (10);
    push (15);
    push (20);
    display();
    pop () ;
    display();
    cout << peek () << endl;
    return 0;
}
// الدالة الخاصة بالاضافة
void push(int value)
{
    if (top == size -1)
    { cout << "Stake Oner Flow. \n";}
    else
    {
        top ++ ;
        stake[top] = value; }}
// الدالة الخاصة بحذف اخر عنصر
int pop ()
{
    if (top == -1)
    { cout << "Stake Under Flwo. \n";}
```

```

    else
    { return stake[top--]; }}
// الدالة الخاصة بعرض اخر عنصر
int peek ()
{
    if (top == -1)
    { cout << "Stake Under Flwo. \n";}
    else
    { return stake[top];}}
// الدالة الخاصة بعرض كل العناصر
void display ()
{
    if (top == -1)
    { cout << "Stake Under Flwo. \n";}
    else
    {
        for (int i = top; i >= 0 ; i--)
        { cout << stake[i] << " : ";}
        cout << "\n-----\n"; }}

```

الدرس الحادي عشر خاص ب ثالث نوع

وهو ال QUEUE

```

/* انواع ال QUEUE
1- SIMPLE QUEUE
2- CIRCULAR QUEUE
3- PRLORLTY QUEUE
4- DEUBLE ENDED QUEUE
*/
#include <iostream>
using namespace std;
// 1- SIMPLE QUEUE
// EN QUEUE FANCTION //! اضافة
// DE QUEUE FANCTION //! حذف
// peek FANCTION      //! عرض اخر عنصر
// display FANCTION   //! عرض كل العناصر
const int size = 5;
int queue [size];
int front = -1;
int rear  = -1;
void EnQueue(int valeu);
void DeQueue();

```



```

int peek ();
void display();
int main()
{
    EnQueue(5);
    EnQueue(10);
    EnQueue(15);
    display();
    DeQueue();
    display();
    cout << "Peek = " << peek() << endl;
    return 0;
}
//*****//
void EnQueue(int valeu)
{
    /* في حالة انه في مكان فاضي
    if (rear != size - 1)
    {
        /* في حالة انه فاضي خالص
        if (rear == -1 && front == -1)
        {
            front++;
            rear ++;
            // queue[front] = valeu;
            queue[rear] = valeu;
        }
        /* في حالة انه فيه عناصر ولكن مش ملين
        else
        {
            rear ++;
            queue[rear] = valeu;
        }
    }
    /* في حالة انه ملين
    else
    {
        cout << "=====\n";
        cout << "== Queue is Full.==\n";
        cout << "=====\n";
    }
}
//*****//
void DeQueue()// حذف
{

```

```

//? في حالة انه فيه عناصر
if (front != -1 && rear != -1 && front <= rear)
{
    front ++;

}
//? في حالة انه فاضي خالص
else
{
    cout << "=====\n";
    cout << "== Queue is Empty. ==\n";
    cout << "=====\n";
}
}
//*****//
int peek ()
{
    if (front == -1 && rear == -1 && front > rear)
    {
        cout << "=====\n";
        cout << "== Queue is Empty. ==\n";
        cout << "=====\n";
        return 0;
    }
    else
        return queue[front];
}
//*****//
void display()
{
    if (front == -1 && rear == -1 && front > rear)
    {
        cout << "=====\n";
        cout << "== Queue is Empty. ==\n";
        cout << "=====\n";
    }
    else
    {
        for (int i = front; i <= rear; i++)
        {
            cout << queue[i] << "\t";
        }
        cout << endl;
    }
}
}

```

الدرس الثاني عشر

الدرس الثالث عشر

الدرس الرابع عشر

الدرس الخامس عشر

الدرس السادس عشر

الدرس السابع عشر

الدرس الثامن عشر

الدرس التاسع عشر

الدرس العشرون

الدرس الحادي والعشرون

الدرس الثاني والعشرون

الدرس الثالث والعشرون

الدرس الرابع والعشرون

الدرس الخامس والعشرون